

Universidad Autónoma de Chiapas

TEMA: Plataforma SaaS Multi-Tenant

Documentación Técnica: API REST, Roles, Catálogo QR y
App Kotlin

Ingeniería en Desarrollo de Software

6 de octubre de 2026

Índice

1. Objetivo del Caso Práctico	3
1.1. Cómo leer este documento	3
1.2. Alcance del proyecto	3
2. Arquitectura General	4
2.1. Pila tecnológica	4
2.2. Estructura de carpetas relevante	4
2.3. Prefijo y versionado de la API	5
2.4. Flujo completo de una petición protegida	6
2.5. Principio de arquitectura desacoplada	6
3. Requerimientos Funcionales	7
3.1. 1. Gestión de Usuarios	7
3.1.1. Registro de usuarios	7
3.1.2. Inicio de sesión y emisión del JWT	9
3.1.3. Estructura de un JWT	10
3.1.4. Cierre de sesión	11
3.1.5. Recuperación de contraseña	11
3.2. 2. Gestión de Roles	11
3.2.1. El seeder paso a paso	11
3.2.2. Matriz de roles y permisos	12
3.2.3. Usuarios de demostración	12
3.2.4. Cómo se vincula un usuario con un rol	13
3.3. 3. Gestión de Permisos	13
3.3.1. Modelo de datos de Spatie	13
3.3.2. Creación dinámica de roles	14
3.4. 4. Control de Acceso y Auditoría	15
3.4.1. El middleware de roles	15
3.4.2. Dónde se aplica	15
3.4.3. Prueba del control de acceso	16
3.4.4. La bitácora de auditoría	16
3.5. 5. Dashboard de Administración	18

3.5.1.	Tabla resumen de endpoints de administración	18
3.5.2.	Detalles de implementación destacables	18
3.6.	6. Arquitectura Web y Consumo de Servicios	19
4.	Requerimientos Técnicos	20
4.1.	Lenguaje, framework y base de datos	20
4.1.1.	Configuración de la base de datos	20
4.1.2.	Migraciones	20
4.1.3.	Acceso a datos y protección contra inyección SQL	21
4.2.	Frontend web	22
4.3.	API REST plena	22
4.3.1.	Verbos HTTP y su semántica	22
4.3.2.	Catálogo completo de endpoints	22
4.3.3.	Respuestas siempre en JSON	23
4.3.4.	Códigos de estado usados	23
4.4.	Autenticación JWT con Laravel Passport	24
4.4.1.	Por qué Passport	24
4.4.2.	Configuración del guard	24
4.4.3.	Claves RSA	24
4.4.4.	Tablas de Passport	25
4.4.5.	Ciclo de vida de un token	25
4.4.6.	Cómo se valida cada petición	25
5.	Requerimientos de Seguridad Computacional	26
5.1.	1. Cifrado de contraseñas	26
5.1.1.	Dónde se aplica	26
5.1.2.	Cómo funciona Bcrypt	26
5.1.3.	Verificación	27
5.2.	2. Comunicación Cifrada (HTTPS/TLS)	27
5.2.1.	Por qué es indispensable	27
5.2.2.	Cómo se implementa en producción	27
5.3.	3. Gestión segura de tokens JWT	28
5.3.1.	Expiración	28

5.3.2.	Firma y permisos de las claves	29
5.3.3.	Revocación	29
5.3.4.	Almacenamiento del token en el cliente	29
5.4.	4. Autorización verificada en el servidor	29
5.4.1.	El principio	29
5.4.2.	Implementación	30
5.4.3.	Prueba de que el ocultamiento visual no basta	30
5.5.	5. Validación y Sanitización de Entradas	30
5.5.1.	Reglas aplicadas por endpoint	30
5.5.2.	Defensas combinadas	31
5.5.3.	Ejemplo de rechazo por validación	31
5.6.	6. Protección contra ataques de fuerza bruta	32
5.6.1.	Dónde está	32
5.6.2.	Cómo funciona	32
5.6.3.	Combinación con otras defensas	32
5.7.	7. Registro de auditoría con trazabilidad	32
5.7.1.	Qué eventos quedan registrados	32
5.7.2.	Cómo consultar la bitácora	33
5.7.3.	Sobre la inmutabilidad y la confidencialidad	33
5.8.	8. Manejo seguro de CORS	34
5.8.1.	Qué es CORS	34
5.8.2.	Configuración	34
5.9.	9. Principio de Mínimo Privilegio	34
5.9.1.	Cómo se aplica en el proyecto	35
5.9.2.	Comparación de capacidades por rol	35
5.10.	10. Manejo de errores sin fuga de información	35
5.10.1.	Por qué importa	35
5.10.2.	Configuración requerida en el archivo .env de producción	36
5.10.3.	Comportamiento esperado	36
6.	Frontend Web y Experiencia de Usuario	36
6.1.	Diseño y Estética (Glassmorphism)	36
6.1.1.	Implementación visual	37

6.2.	Estructura Lógica del Cliente	37
6.3.	Consumo Seguro de la API REST	37
6.3.1.	Gestión del Token en el cliente	37
6.3.2.	Prevención de XSS en la construcción del DOM	38
6.4.	Manejo de Errores y Respuesta visual	38
6.5.	Pruebas de la API vía CLI	39
7.	Arquitectura SaaS Multi-Tenant y App Móvil Kotlin	39
7.1.	1. Objetivo General de la Plataforma	39
7.2.	2. Jerarquía de Roles y Matriz de Control de Acceso	39
7.3.	3. Modelo de Datos y Diagrama de Tablas MySQL	40
7.4.	4. Comparativa de Tecnologías Utilizadas	41
7.5.	5. Flujo de Integración API con App Móvil Kotlin	42

1. Objetivo del Caso Práctico

El objetivo es desarrollar una aplicación web que implemente autenticación y autorización basada en roles (RBAC), consumiendo una API REST protegida mediante JSON Web Tokens (JWT). Se garantiza que los usuarios accedan a diferentes funcionalidades según su rol, asegurando la confidencialidad e integridad de la información en todo momento.

Solución Tecnológica: Se eligió **Laravel** (PHP) como motor backend y un frontend desacoplado desarrollado en HTML/CSS/JS (Vanilla).

1.1. Cómo leer este documento

Este documento no solo enumera los requerimientos del caso práctico, sino que explica **cómo** y **dónde** se implementó cada uno de ellos dentro del proyecto `llm-chat`. Para cada requerimiento se presenta:

1. **Qué se pidió:** el enunciado del requerimiento.
2. **Dónde está:** el archivo y, cuando aplica, la clase o método del proyecto que lo resuelve.
3. **Cómo funciona:** el fragmento de código real y una explicación paso a paso.
4. **Cómo comprobarlo:** comandos o peticiones HTTP que permiten verificar el comportamiento.

Al final (capítulo ??) se incluye una matriz de cumplimiento y una sección de observaciones honestas sobre qué partes están completamente implementadas, cuáles dependen de la configuración del servidor de producción y cuáles quedaron pendientes.

1.2. Alcance del proyecto

El proyecto `llm-chat` es el backend de una aplicación de chat con un modelo de lenguaje (LLM). En esta etapa se construyó la **capa de identidad y control de acceso**, que es la base sobre la cual se montarán las funciones de conversación:

- Registro e inicio de sesión de usuarios.
- Emisión y validación de tokens JWT.
- Roles y permisos administrables.
- Panel de administración (frontend desacoplado).
- Bitácora de auditoría de acciones.

Los endpoints de conversaciones y de envío de mensajes al LLM ya están declarados en las rutas y protegidos por autenticación, pero su lógica de negocio devuelve por ahora una respuesta marcada como (*Pendiente*).

2. Arquitectura General

2.1. Pila tecnológica

Componente	Tecnología y función
Framework backend	Laravel (PHP). Define rutas, middlewares, ORM y validación. Versión declarada en <code>composer.json</code> : <code>laravel/framework ^12.0</code> .
Autenticación	Laravel Passport ^13.8 (servidor OAuth2). Emite tokens de acceso firmados con RSA.
Roles y permisos	Spatie Laravel Permission ^8.3. Tablas relacionales para roles, permisos y sus asignaciones.
Base de datos	SQLite en desarrollo (<code>database/database.sqlite</code>); MySQL compatible gracias al ORM Eloquent.
Frontend	HTML, CSS y JavaScript sin frameworks (<code>public/admin.html</code>).
Servidor de desarrollo	<code>php artisan serve</code> .
Documentación	L ^A T _E X (este documento).

Sobre la versión de Laravel

Una versión previa de esta guía mencionaba Laravel 11. El archivo `composer.json` del proyecto declara `^12.0`, por lo que la versión real instalada es la 12. La estructura que se describe aquí (archivo `bootstrap/app.php` como punto de configuración de middlewares y rutas) es la misma que introdujo Laravel 11 y se mantiene en la 12.

2.2. Estructura de carpetas relevante

La siguiente tabla indica qué archivo del proyecto resuelve cada responsabilidad. Se recomienda tenerla presente al leer los capítulos siguientes.

Archivo	Responsabilidad
<code>bootstrap/app.php</code>	Registra el prefijo de la API (<code>api/v1</code>), el alias de middleware <code>role</code> y el middleware de auditoría.
<code>routes/api.php</code>	Define todos los endpoints REST y qué middleware protege a cada grupo.
<code>routes/web.php</code>	Solo redirige <code>/</code> y <code>/admin</code> hacia <code>admin.html</code> . No renderiza vistas Blade.
<code>app/Http/Controllers/Api/AuthController.php</code>	Registra <code>login</code> y <code>logout</code> .

Archivo	Responsabilidad
app/Http/Controllers/Api/AdminController.php	Controlador de roles, asignación de roles, creación de roles y consulta de auditoría.
app/Http/Middleware/RoleMiddleware.php	Verifica que el usuario autenticado tenga alguno de los roles exigidos; si no, responde 403.
app/Http/Middleware/AuditMiddleware.php	Registra en bitácora cada petición hecha por un usuario autenticado.
app/Models/User.php	Modelo de usuario con los <i>traits</i> <code>HasApiTokens</code> (Passport) y <code>HasRoles</code> (Spatie).
app/Models/AuditLog.php	Modelo de la tabla <code>audit_logs</code> .
database/seeder/RolesAndPermissionsSeeder.php	Clase <code>Seeder</code> que crea roles y dos usuarios de demostración.
database/migrations/	Estructura de tablas (usuarios, Passport, Spatie, auditoría).
config/auth.php	Declara el guard <code>api</code> con driver <code>passport</code> .
config/permission.php	Configuración de Spatie (nombres de tablas, caché).
storage/oauth-private.key oauth-public.key	Par de claves RSA con las que Passport firma y verifica los tokens.
public/admin.html	Dashboard de administración (cliente de la API).

2.3. Prefijo y versionado de la API

En `bootstrap/app.php` se configura `apiPrefix: 'api/v1'`. Por esa razón, aunque en `routes/api.php` una ruta se declare como `/login`, la URL pública real es `/api/v1/login`. Incluir la versión (v1) en la URL permite publicar en el futuro una v2 sin romper a los clientes existentes.

```
return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',
        apiPrefix: 'api/v1',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        $middleware->alias([
            'role' => \App\Http\Middleware\RoleMiddleware::class,
        ]);

        $middleware->api(append: [
            \App\Http\Middleware\AuditMiddleware::class,
        ]);
    })
    ->withExceptions(function (Exceptions $exceptions): void {
```



```
//
})->create();
```

Listing 1: Configuración de rutas y middlewares en bootstrap/app.php

Puntos clave de este archivo:

- `alias(['role' =>...])`: crea la palabra clave `role` que luego se usa en las rutas como `middleware('role:Administrador')`.
- `api(append: [...])`: agrega el `AuditMiddleware` al final del grupo de middlewares `api`, es decir, se ejecuta en toda ruta de `routes/api.php`.
- `health: '/up'`: expone un endpoint de salud que sirve para comprobar que la aplicación responde.

2.4. Flujo completo de una petición protegida

El siguiente esquema muestra, en orden, lo que ocurre cuando el dashboard solicita la lista de usuarios (GET /api/v1/admin/users):

```
Navegador (admin.html)
| GET /api/v1/admin/users
| Authorization: Bearer <JWT>
v
[1] public/index.php -> arranca Laravel
[2] Prefijo api/v1 -> se selecciona routes/api.php
[3] Middleware del grupo "api" (incluye AuditMiddleware)
[4] auth:api -> Passport valida firma RS256, expiracion
    y que el token no este revocado
[5] role:Administrador-> RoleMiddleware consulta a Spatie
    |-- sin rol -> 403 {"message":"Forbidden - ..."}
    v
[6] AdminController@listUsers -> User::with('roles')->get()
    |
    v
Respuesta JSON 200 (lista de usuarios con sus roles)
```

Listing 2: Ciclo de vida de una petición protegida

Si en el paso [4] el token es inválido, está vencido o fue revocado, Laravel responde **401 Unauthorized** y la petición nunca llega al controlador. Si el token es válido pero el rol es insuficiente, el paso [5] responde **403 Forbidden**. La diferencia entre 401 (*no sé quién eres*) y 403 (*sé quién eres, pero no puedes hacer esto*) es fundamental en el diseño de APIs seguras.

2.5. Principio de arquitectura desacoplada

El backend no genera HTML. Todo lo que el dashboard muestra proviene de respuestas JSON. Esto trae tres ventajas directas para la seguridad:

1. **Superficie de ataque reducida:** no hay plantillas del servidor que puedan inyectar contenido sin escapar.
2. **Sin estado de sesión en el servidor:** cada petición lleva su propia credencial (el token), por lo que no existe una cookie de sesión que pueda ser robada mediante CSRF.
3. **Reemplazabilidad del cliente:** el mismo backend puede servir a una app móvil o a otro frontend sin cambios. De hecho, el comentario de `routes/api.php` indica que las rutas públicas de autenticación fueron pensadas para una aplicación móvil además del dashboard web.

3. Requerimientos Funcionales

3.1. 1. Gestión de Usuarios

Se crearon los endpoints en `AuthController.php`:

- **Registro:** POST `/api/v1/register` (Nombre, Email, Contraseña).
- **Inicio de Sesión:** POST `/api/v1/login` que devuelve el JWT.
- **Cierre de sesión:** POST `/api/v1/logout`, que revoca el token en uso.
- **Recuperación de Contraseña:** La arquitectura de base de datos soporta los flujos de `password_resets` integrados en Laravel.

3.1.1. Registro de usuarios

Dónde: método `register` de `app/Http/Controllers/Api/AuthController.php`. La ruta es pública (no requiere token) porque un usuario nuevo todavía no puede tener uno.

```
public function register(Request $request)
{
    $request->validate([
        'name' => 'required|string|max:255',
        'email' => 'required|email|unique:users',
        'password' => 'required|min:8',
    ]);

    $user = User::create([
        'name' => $request->name,
        'email' => $request->email,
        'password' => Hash::make($request->password),
    ]);

    // Default role for new users
    $user->assignRole('Usuario Regular');

    AuditLog::create([
```

```
        'user_id' => $user->id,  
        'action' => 'Registro de usuario',  
        'ip_address' => $request->ip(),  
        'details' => 'Nuevo usuario registrado vía API'  
    ]);  
  
    return response()->json(['message' => 'Usuario registrado  
        exitosamente',  
                             'user' => $user], 201);  
}
```

Listing 3: AuthController::register

Explicación paso a paso:

1. **Validación.** `$request->validate()` comprueba que el nombre exista y tenga máximo 255 caracteres, que el correo tenga formato válido y no esté repetido en la tabla `users` (regla `unique:users`) y que la contraseña tenga al menos 8 caracteres. Si algo falla, Laravel corta la ejecución y responde **422 Unprocessable Entity** con el detalle de cada campo inválido; el código siguiente no se ejecuta.
2. **Creación del usuario.** Se guarda con `Hash::make`, de modo que en la base de datos nunca queda la contraseña original (ver sección de cifrado).
3. **Rol por defecto.** Todo usuario que se registra por sí mismo recibe el rol `Usuario Regular`, el de menores privilegios. Esto es una aplicación directa del principio de mínimo privilegio: el registro público **nunca** puede crear administradores.
4. **Auditoría.** Se inserta un registro con el ID del nuevo usuario, la IP desde la que se registró y una descripción.
5. **Respuesta 201 Created.** Se devuelve el usuario. Como el modelo declara `$hidden = ['password', 'remember_token']`, esos campos se omiten automáticamente del JSON.

Ejemplo de uso:

```
curl -X POST http://127.0.0.1:8000/api/v1/register \  
-H "Accept: application/json" \  
-H "Content-Type: application/json" \  
-d '{"name":"Ana","email":"ana@example.com","password":"  
claveSegura123"}'
```

Listing 4: Registro desde la línea de comandos

```
{  
  "message": "Usuario registrado exitosamente",  
  "user": { "id": 3, "name": "Ana", "email": "ana@example.com",  
            "updated_at": "...", "created_at": "..." }  
}
```

Listing 5: Respuesta esperada (201)

3.1.2. Inicio de sesión y emisión del JWT

Dónde: método login de AuthController. La ruta está protegida contra fuerza bruta con `throttle:5,1`.

```
public function login(Request $request)
{
    $request->validate([
        'email' => 'required|email',
        'password' => 'required',
    ]);

    $user = User::where('email', $request->email)->first();

    if (! $user || ! Hash::check($request->password, $user->password))
    {
        return response()->json(['message' => 'Credenciales incorrectas'], 401);
    }

    AuditLog::create([
        'user_id' => $user->id,
        'action' => 'Inicio de sesión',
        'ip_address' => $request->ip(),
        'details' => 'Inicio de sesión exitoso'
    ]);

    $tokenResult = $user->createToken('Personal Access Token');
    $token = $tokenResult->token;
    $token->expires_at = now()->addMinutes(720); // 12 horas
    $token->save();

    return response()->json([
        'access_token' => $tokenResult->accessToken,
        'token_type' => 'Bearer',
        'expires_at' => \Carbon\Carbon::parse($tokenResult->token->expires_at)
                                ->toDateTimeString(),
        'role' => $user->getRoleNames()->first()
    ]);
}
```

Listing 6: AuthController::login

Explicación paso a paso:

1. Se validan formato de correo y presencia de contraseña.
2. Se busca al usuario por correo. Se usa `where()->first()` del ORM, que genera una sentencia SQL preparada (sin concatenar texto), por lo que no es vulnerable a inyección SQL.
3. `Hash::check` compara la contraseña recibida contra el hash Bcrypt almacenado. La comparación se hace en tiempo seguro dentro de la función de PHP.
4. Si el usuario no existe o la contraseña es incorrecta, se devuelve exactamente el mismo mensaje (*Credenciales incorrectas*) con código 401. Esto evita la **enumeración**

de usuarios: un atacante no puede saber si un correo está registrado.

5. Se registra el inicio de sesión en la auditoría.
6. `createToken` genera un token de acceso personal de Passport. El proyecto lo extiende a **12 horas** (720 minutos) modificando `expires_at`.
7. Se responde con el token, su tipo (`Bearer`), su fecha de expiración y el rol del usuario, que el frontend usa para decidir qué mostrar.

Ejemplo de uso y respuesta:

```
curl -X POST http://127.0.0.1:8000/api/v1/login \
  -H "Accept: application/json" -H "Content-Type: application/json" \
  -d '{"email":"admin@admin.com","password":"password123"}'
```

Listing 7: Login y obtención del token

```
{
  "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9....",
  "token_type": "Bearer",
  "expires_at": "2026-10-06 11:30:00",
  "role": "Administrador"
}
```

Listing 8: Respuesta esperada (200)

3.1.3. Estructura de un JWT

El valor de `access_token` es un JWT, formado por tres partes separadas por puntos:

Parte	Contenido
Encabezado (<i>header</i>)	Algoritmo de firma (<code>RS256</code>) y tipo de token (<code>JWT</code>).
Carga (<i>payload</i>)	Identificador del token (<code>jti</code>), identificador del cliente (<code>aud</code>), usuario (<code>sub</code>), fechas de emisión y expiración (<code>iat</code> , <code>exp</code>) y alcances (<code>scopes</code>).
Firma (<i>signature</i>)	Resultado de firmar encabezado y carga con la clave privada RSA. Cualquiera puede leer el contenido, pero nadie puede modificarlo sin invalidar la firma.

Importante sobre confidencialidad

Un JWT está *firmado*, no *cifrado*: su contenido puede decodificarse en Base64 por cualquiera que lo tenga. Por eso el proyecto no incluye contraseñas ni datos sensibles en él, y por eso es indispensable transportarlo siempre por HTTPS.

3.1.4. Cierre de sesión

```
public function logout(Request $request)
{
    AuditLog::create([
        'user_id' => $request->user()->id,
        'action' => 'Cierre de sesión',
        'ip_address' => $request->ip(),
        'details' => 'Cierre de sesión manual'
    ]);

    $request->user()->token()->revoke();
    return response(['message' => 'Sesión cerrada correctamente'], 200);
}
```

Listing 9: AuthController::logout

`token()->revoke()` marca el token como revocado en la tabla `oauth_access_tokens`. A partir de ese momento, aunque el token no haya vencido, Passport lo rechazará con 401. Esto es lo que da sentido real al botón *Cerrar sesión*: no basta con borrar el token del navegador, también se invalida en el servidor.

3.1.5. Recuperación de contraseña

El proyecto incluye las migraciones base de Laravel (`0001_01_01_000000_create_users_table.php`), que crean la tabla `password_reset_tokens` y la tabla `sessions`. Asimismo, en `app/Http/Controllers/` existen los controladores estándar de Laravel Breeze (`PasswordResetLinkController`, `NewPasswordController`, etc.). Por tanto, la *base de datos y la lógica* del flujo de recuperación están disponibles; no obstante, este flujo está pensado para la interfaz web tradicional y no está expuesto como endpoint de la API `api/v1`.

3.2. 2. Gestión de Roles

Se integró el paquete **Spatie Laravel Permission** para gestionar de manera dinámica los tres roles predeterminados:

- **Administrador:** Tiene acceso total (Lectura, Escritura, Eliminación).
- **Editor:** Tiene permisos limitados (Lectura, Escritura).
- **Usuario Regular:** Solo tiene permiso de lectura.

Estos fueron insertados en la base de datos a través de `RolesAndPermissionsSeeder.php`.

3.2.1. El seeder paso a paso

```
// create permissions
Permission::firstOrCreate(['name' => 'Lectura', 'guard_name' => 'api']);
```

```

Permission::firstOrCreate(['name' => 'Escritura', 'guard_name' => 'api']);
Permission::firstOrCreate(['name' => 'Eliminacion', 'guard_name' => 'api']);

// create roles and assign created permissions
$roleRegular = Role::firstOrCreate(['name' => 'Usuario Regular', 'guard_name' => 'api'])
->givePermissionTo(['Lectura']);

$roleEditor = Role::firstOrCreate(['name' => 'Editor', 'guard_name' => 'api'])
->givePermissionTo(['Lectura', 'Escritura']);

$roleAdmin = Role::firstOrCreate(['name' => 'Administrador', 'guard_name' => 'api'])
->givePermissionTo(['Lectura', 'Escritura', 'Eliminacion']);

```

Listing 10: RolesAndPermissionsSeeder::run (fragmento)

- **firstOrCreate** hace que el seeder sea **idempotente**: se puede ejecutar varias veces sin duplicar roles ni permisos.
- **guard_name => 'api'** es esencial. Spatie asocia cada rol y permiso a un *guard*. Como el **RoleMiddleware** se ejecuta tras autenticar con el guard **api** (Passport), los roles deben pertenecer a ese guard; de lo contrario **hasAnyRole** no los encontraría.
- Al inicio del seeder se llama a **forgetCachedPermissions()** para vaciar la caché de Spatie y evitar que datos antiguos interfieran.

3.2.2. Matriz de roles y permisos

Rol	Lectura	Escritura	Eliminación
Usuario Regular	Sí	No	No
Editor	Sí	Sí	No
Administrador	Sí	Sí	Sí

3.2.3. Usuarios de demostración

El seeder también crea dos cuentas para probar el sistema:

Correo	Rol	Uso
admin@admin.com	Administrador	Acceso total al dashboard
editor@editor.com	Editor	Prueba de rechazo 403 en rutas de admin

Credenciales de demostración

Estas cuentas usan una contraseña de ejemplo escrita en el seeder. Son válidas solo para desarrollo y evaluación. En un despliegue real deben eliminarse o cambiarse antes de publicar la aplicación.

3.2.4. Cómo se vincula un usuario con un rol

El modelo `User` usa el *trait* `HasRoles` de Spatie:

```
use Laravel\Passport\HasApiTokens;
use Spatie\Permission\Traits\HasRoles;

class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable, HasRoles;

    protected $fillable = ['name', 'email', 'password'];
    protected $hidden    = ['password', 'remember_token'];

    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password'         => 'hashed',
        ];
    }
}
```

Listing 11: `app/Models/User.php` (fragmento)

Este *trait* añade métodos como `assignRole()`, `syncRoles()`, `hasAnyRole()`, `getRoleNames()` y `givePermissionTo()`. Observe además dos medidas de seguridad en el modelo:

- `$fillable` limita qué campos pueden asignarse en masa; el campo de rol no puede inyectarse mediante *mass assignment*.
- El *cast* `'password' => 'hashed'` indica a Eloquent que cifre la contraseña automáticamente al asignarla.

3.3. 3. Gestión de Permisos

Los permisos (*Lectura*, *Escritura*, *Eliminación*) están desacoplados del código fuente y residen en tablas relacionales (`role_has_permissions`). Esto permite asignar permisos a los roles de forma dinámica y aplicar políticas a áreas específicas de la aplicación.

3.3.1. Modelo de datos de Spatie

La migración `2026_10_05_021919_create_permission_tables.php` crea cinco tablas:

Tabla	Contenido
permissions	Catálogo de permisos (id, name, guard_name).
roles	Catálogo de roles (id, name, guard_name).
role_has_permissions	Relación muchos-a-muchos: qué permisos tiene cada rol.
model_has_roles	Relación muchos-a-muchos: qué roles tiene cada usuario.
model_has_permissions	Permisos asignados directamente a un usuario (sin pasar por rol).

```

users --< model_has_roles >-- roles --< role_has_permissions >--
permissions
\
'-----< model_has_permissions >-----'
/

```

Listing 12: Relaciones entre las tablas

Gracias a este diseño, **cambiar lo que puede hacer un rol no requiere modificar código**: basta con insertar o borrar filas en `role_has_permissions`. El endpoint `POST /admin/roles` permite precisamente crear roles nuevos con la lista de permisos que se desee.

3.3.2. Creación dinámica de roles

```

public function createRole(Request $request)
{
    $request->validate([
        'name' => 'required|string|unique:roles,name',
        'permissions' => 'array',
        'permissions.*' => 'string|exists:permissions,name'
    ]);

    $role = Role::create(['name' => $request->name, 'guard_name' => 'api']);

    if ($request->has('permissions')) {
        $role->givePermissionTo($request->permissions);
    }

    AuditLog::create([ /* user_id, 'Crear Rol', ip_address, details */
    ]);

    return response()->json(['message' => 'Rol creado exitosamente',
        'role' => $role], 201);
}

```

Listing 13: AdminController::createRole

La validación `permissions.*: exists:permissions,name` garantiza que solo se puedan asociar permisos que ya existen en el catálogo, evitando referencias inválidas.

```

curl -X POST http://127.0.0.1:8000/api/v1/admin/roles \
-H "Authorization: Bearer $TOKEN" -H "Accept: application/json" \

```

```
-H "Content-Type: application/json" \  
-d '{"name":"Moderador","permissions":["Lectura","Escritura"]}'
```

Listing 14: Crear el rol "Moderador" con permisos de lectura y escritura

3.4. 4. Control de Acceso y Auditoría

El acceso está controlado desde el backend usando `Route::middleware('role:Administrador')`. Si un Editor intenta acceder a una ruta de Administrador, el servidor responde con **403 Forbidden**.

Historial de Acceso: Se creó el modelo `AuditLog` en Laravel. En cada acción (registro, login, asignar rol), se captura la dirección IP del usuario, ID, y acción, guardándola de forma inmutable.

3.4.1. El middleware de roles

```
public function handle(Request $request, Closure $next, string ...  
    $roles): Response  
{  
    if (! $request->user() || ! $request->user()->hasAnyRole($roles)) {  
        return response()->json(['message' => 'Forbidden - Insufficient  
            permissions'], 403);  
    }  
  
    return $next($request);  
}
```

Listing 15: `app/Http/Middleware/RoleMiddleware.php`

- Recibe los roles permitidos como parámetros variables: `role:Administrador` o, por ejemplo, `role:Administrador,Editor`.
- `hasAnyRole($roles)` consulta las tablas de Spatie. Es una verificación hecha **siempre en el servidor**, con datos de la base de datos, no con datos que envíe el cliente.
- Si no hay usuario autenticado, o no posee ninguno de los roles, responde 403 y la petición se detiene.

3.4.2. Dónde se aplica

```
// Rutas protegidas (Requieren JWT de Passport)  
Route::middleware('auth:api')->group(function () {  
  
    // ... rutas de perfil, conversaciones y chat ...  
  
    // Rutas de Administración (Protegidas por middleware de rol)  
    Route::middleware('role:Administrador')->group(function () {  
        Route::get('/admin/users', [AdminController::class, 'listUsers'  
            ]);  
    });  
});
```

```

Route::post('/admin/users', [AdminController::class, '
    createUser']);
Route::put('/admin/users/{userId}', [AdminController::class, '
    updateUser']);
Route::delete('/admin/users/{userId}', [AdminController::class, '
    deleteUser']);
Route::post('/admin/users/{userId}/role', [AdminController::
    class, 'assignRole']);
Route::post('/admin/roles', [AdminController::class, '
    createRole']);
Route::get('/admin/audit-logs', [AdminController::class, '
    auditLogs']);
});

Route::post('/logout', [AuthController::class, 'logout']);
});

```

Listing 16: routes/api.php (fragmento)

Los grupos están **anidados**: primero se exige un token válido (`auth:api`) y, dentro de ese grupo, se exige además el rol de administrador. Una ruta de administración, por tanto, atraviesa dos filtros independientes.

3.4.3. Prueba del control de acceso

```

# 1) Login como editor
TOKEN=$(curl -s -X POST http://127.0.0.1:8000/api/v1/login \
-H "Accept: application/json" -H "Content-Type: application/json" \
-d '{"email":"editor@editor.com","password":"password123"}' \
| php -r 'echo json_decode(stream_get_contents(STDIN))->access_token;
')

# 2) Intento de acceder a una ruta de administrador
curl -i http://127.0.0.1:8000/api/v1/admin/users \
-H "Authorization: Bearer $TOKEN" -H "Accept: application/json"

```

Listing 17: Un Editor intenta listar usuarios

```

HTTP/1.1 403 Forbidden
{"message":"Forbidden - Insufficient permissions"}

```

Listing 18: Respuesta

3.4.4. La bitácora de auditoría

Estructura de la tabla (migración `create_audit_logs_table`):

```

Schema::create('audit_logs', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->nullable()->constrained('users')->
        nullOnDelete();
    $table->string('action');
    $table->string('ip_address')->nullable();
    $table->text('details')->nullable();
});

```

```
$table->timestamps();
});
```

Listing 19: Migración de audit_logs

Columna	Propósito
id	Identificador autoincremental.
user_id	Quién realizó la acción. Es llave foránea a <code>users</code> ; con <code>onDelete</code> el registro se conserva aunque el usuario sea eliminado (el campo pasa a NULL), de modo que la historia no se pierde.
action	Qué se hizo (por ejemplo <i>Inicio de sesión</i> o <i>POST api/v1/admin/roles</i>).
ip_address	Desde qué dirección IP.
details	Información adicional en texto.
created_at	Cuándo ocurrió (marca de tiempo automática).

Dos mecanismos de registro complementarios:

1. **Registro explícito en controladores.** Los eventos de negocio relevantes (registro, login, logout, crear/editar/eliminar usuario, asignar rol, crear rol) llaman a `AuditLog::create(...)` con una descripción legible.
2. **Registro automático por middleware.** `AuditMiddleware` anota cada petición realizada por un usuario autenticado:

```
public function handle(Request $request, Closure $next): Response
{
    if ($request->user()) {
        AuditLog::create([
            'user_id' => $request->user()->id,
            'action' => $request->method() . ' ' . $request->path(),
            'ip_address' => $request->ip(),
            'details' => json_encode(
                $request->except(['password', 'password_confirmation'])
            ),
        ]);
    }

    return $next($request);
}
```

Listing 20: `app/Http/Middleware/AuditMiddleware.php`

Se observa que, antes de guardar el cuerpo de la petición en `details`, se **excluyen los campos de contraseña**. Así la propia bitácora no se convierte en una fuga de credenciales.

Sobre la inmutabilidad. La aplicación no expone ningún endpoint para modificar ni borrar registros de auditoría: solo existe `GET /admin/audit-logs` (lectura) y la inserción

interna desde el código. En ese sentido, ningún usuario puede alterar la bitácora a través de la API. Esta garantía es *a nivel de aplicación*; ver el capítulo ?? para las recomendaciones de reforzarla a nivel de base de datos.

3.5. 5. Dashboard de Administración

El `AdminController.php` expone las siguientes rutas protegidas exclusivas para administradores:

- Ver usuarios registrados (GET `/admin/users`).
- Crear, editar y eliminar usuarios (POST, PUT, DELETE).
- Asignar/revocar roles (POST `/admin/users/{id}/role`).
- Crear roles (POST `/admin/roles`).
- Consultar registros de auditoría (GET `/admin/audit-logs`).

Recuerde que todas llevan el prefijo `/api/v1`.

3.5.1. Tabla resumen de endpoints de administración

Verbo	Ruta	Acción y validaciones
GET	<code>/admin/users</code>	Lista todos los usuarios con sus roles (<code>User::with('roles')</code>).
POST	<code>/admin/users</code>	Crea usuario. Valida nombre, correo único, contraseña mínimo 8 y que el rol exista.
PUT	<code>/admin/users/{userId}</code>	Actualiza nombre, correo (único, ignorando al propio usuario) y/o rol.
DELETE	<code>/admin/users/{userId}</code>	Elimina usuario. Un administrador no puede eliminarse a sí mismo (400).
POST	<code>/admin/users/{userId}/role</code>	Reemplaza los roles del usuario por uno nuevo (<code>syncRoles</code>).
POST	<code>/admin/roles</code>	Crea un rol y opcionalmente sus permisos.
GET	<code>/admin/audit-logs</code>	Devuelve la bitácora, más reciente primero (<code>AuditLog::latest()</code>).

3.5.2. Detalles de implementación destacables

Eliminación segura.

```
public function deleteUser(Request $request, $id)
{
    $user = User::findOrFail($id);

    if ($user->id === $request->user()->id) {
        return response()->json(['message' => 'No puedes eliminarte a ti mismo'], 400);
    }

    $user->delete();
    AuditLog::create([ /* 'Eliminar Usuario', IP, detalle */ ]);
    return response()->json(['message' => 'Usuario eliminado correctamente']);
}
```

Listing 21: AdminController::deleteUser

`findOrFail` devuelve **404** si el usuario no existe. La comparación con `$request->user()->id` evita que el único administrador se elimine por error y deje el sistema sin nadie capaz de gestionarlo.

Actualización con unicidad que ignora al propio registro. En `updateUser` la regla `unique:users,email,$id` indica que el correo debe ser único *excepto* para el usuario que se está editando; así es posible guardar un perfil sin cambiar el correo sin recibir un falso error de duplicado. Solo se actualizan `name` y `email` mediante `$request->only(...)`, por lo que otros campos enviados en la petición se ignoran.

Reemplazo de rol. Tanto `assignRole` como `updateUser` usan `syncRoles(...)`, que *sustituye* los roles anteriores. Cada usuario administrado desde el panel queda, por tanto, con un único rol, lo que simplifica el razonamiento sobre sus privilegios.

3.6. 6. Arquitectura Web y Consumo de Servicios

Se aplicó una **arquitectura desacoplada estricta**:

- **Frontend:** Es un cliente tonto (`public/admin.html`) que no tiene acceso a la base de datos.
- **Backend:** Expone todo exclusivamente a través de la API REST (`routes/api.php`).
- **JWT:** La comunicación viaja en los headers como `Authorization: Bearer <token>`. No se usan Vistas de Blade para el renderizado del dashboard para no romper la regla del estado (Stateless).

Cómo se materializa en el proyecto:

- `routes/web.php` contiene únicamente dos redirecciones hacia `/admin.html`; no hay ninguna ruta que devuelva una vista Blade para el panel.
- El archivo `public/admin.html` es un documento estático servido directamente por el servidor web. Toda interacción con datos se hace con `fetch()` hacia `/api/v1`.

- El frontend no contiene credenciales de base de datos ni lógica de autorización; solo conserva el token que el servidor le entregó.

En el capítulo 6 se detalla el funcionamiento interno del dashboard.

4. Requerimientos Técnicos

1. **Lenguaje y SMBD:** PHP con Laravel. Motor de base de datos MySQL/SQLite a través del ORM Eloquent.
2. **Frontend Web:** Aplicación accesible desde navegador usando HTML/CSS/JS avanzado con diseño Glassmorphism e inyecciones vía AJAX.
3. **API REST Plena:** Se utilizaron los verbos HTTP GET, POST, PUT, DELETE. Todas las respuestas devuelven `application/json`.
4. **Autenticación JWT:** El inicio de sesión usa **Laravel Passport** (OAuth2). Valida credenciales, firma el JWT y el middleware `auth:api` se encarga de interceptar y validar automáticamente la firma en peticiones subsecuentes.

En las secciones siguientes se detalla cómo está construido cada uno de estos puntos.

4.1. Lenguaje, framework y base de datos

4.1.1. Configuración de la base de datos

La conexión se define en el archivo de entorno `.env` (que **no** se versiona) mediante la variable `DB_CONNECTION`. En el entorno de desarrollo se usa `sqlite`, con el archivo `database/database.sqlite`. Para pasar a MySQL basta cambiar las variables de conexión (`DB_CONNECTION=mysql`, `DB_HOST`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`) sin tocar una sola línea de código, porque todo el acceso se realiza a través de Eloquent y del constructor de consultas de Laravel.

4.1.2. Migraciones

La estructura de la base se versiona en `database/migrations/`. Cada archivo es una clase con un método `up()` (crear) y otro `down()` (revertir). Se ejecutan con:

```
php artisan migrate
php artisan db:seed --class=RolesAndPermissionsSeeder
```

Listing 22: Crear tablas y cargar datos iniciales

Migración	Tablas que crea o modifica
0001_01_01_000000_create_users_table	users, password_reset_tokens, sessions.
0001_01_01_000001_create_cache_table	cache, cache_locks.
0001_01_01_000002_create_jobs_table	jobs, job_batches, failed_jobs.
2026_10_05_012439_create_personal_access_tokens_table	personal_access_tokens (Sanctum, instalado inicialmente).
2026_10_05_014524...014528_create_oauth_auth_codes, oauth_access_tokens, oauth_refresh_tokens, oauth_clients, oauth_device_codes	(Passport).
2026_10_05_015658_add_role_to_users_table	Creación de una role (enum) en users. Ver observaciones.
2026_10_05_015659_create_audit_logs_table	audit_logs.
2026_10_05_021919_create_permission_tables	Tablas de Spatie (roles y permisos).

Evolución del proyecto

El orden cronológico de las migraciones refleja cómo evolucionó el diseño: primero se instaló Sanctum (**personal_access_tokens**), luego se migró a Passport (tablas **oauth_***) para obtener firmas JWT con RSA, y finalmente se reemplazó la columna simple **role** por el sistema completo de Spatie. La columna **role** y la tabla de Sanctum permanecen en el esquema pero la lógica actual no las utiliza.

4.1.3. Acceso a datos y protección contra inyección SQL

Ninguna consulta del proyecto concatena texto proporcionado por el usuario dentro de SQL. Todos los accesos usan Eloquent (`User::where(...)`, `User::findOrFail(...)`, `AuditLog::create(...)`), que internamente emplea *sentencias preparadas* con parámetros enlazados (PDO). Ejemplo:

```
$user = User::where('email', $request->email)->first();

-- SQL generado (el valor viaja como parametro, no como texto SQL):
select * from "users" where "email" = ? limit 1
```

Listing 23: Consulta del login y SQL equivalente generado

Aunque un atacante envíe como correo ' OR 1=1 -, el valor se trata como un simple dato y no como parte de la instrucción; además la regla **email** de la validación ya lo rechazaría por formato.

4.2. Frontend web

El frontend es una aplicación de una sola página (*SPA*) en `public/admin.html` con estilos propios (efecto *glassmorphism*: paneles translúcidos con desenfoque de fondo) y JavaScript puro. Las operaciones se hacen con AJAX mediante `fetch()`, sin recargar la página. El capítulo 6 describe su funcionamiento.

4.3. API REST plena

4.3.1. Verbos HTTP y su semántica

Verbo	Semántica	Uso en el proyecto
GET	Leer, sin efectos secundarios.	<code>/user</code> , <code>/admin/users</code> , <code>/admin/audit-logs</code> .
POST	Crear o ejecutar una acción.	<code>/register</code> , <code>/login</code> , <code>/logout</code> , <code>/admin/users</code> , <code>/admin/roles</code> .
PUT	Actualizar un recurso existente.	<code>/admin/users/{id}</code> .
DELETE	Eliminar un recurso.	<code>/admin/users/{id}</code> .

4.3.2. Catálogo completo de endpoints

La salida del comando `php artisan route:list -path=api` muestra los 16 endpoints registrados:

Verbo	URL	Acceso	Estado
POST	<code>/api/v1/register</code>	Público	Implementado
POST	<code>/api/v1/login</code>	Público (limitado 5/min)	Implementado
POST	<code>/api/v1/logout</code>	Token	Implementado
GET	<code>/api/v1/user</code>	Token	Devuelve el usuario autenticado
PUT	<code>/api/v1/user/profile</code>	Token	Marcador (pendiente)
GET	<code>/api/v1/conversations</code>	Token	Marcador (pendiente)
POST	<code>/api/v1/conversations</code>	Token	Marcador (pendiente)
GET	<code>/api/v1/conversations/{id}/messages</code>	Token	Marcador (pendiente)
POST	<code>/api/v1/chat/send</code>	Token	Marcador (pendiente)
GET	<code>/api/v1/admin/users</code>	Token + Administrador	Implementado

Verbo	URL	Acceso	Estado
POST	/api/v1/admin/users	Token + Administrador	Implementado
PUT	/api/v1/admin/users/{userId}	Token + Administrador	Implementado
DELETE	/api/v1/admin/users/{userId}	Token + Administrador	Implementado
POST	/api/v1/admin/users/{userId}/Token	Token + Administrador	Implementado
POST	/api/v1/admin/roles	Token + Administrador	Implementado
GET	/api/v1/admin/audit-logs	Token + Administrador	Implementado

4.3.3. Respuestas siempre en JSON

Los controladores devuelven siempre `response()->json(...)`, que fija el encabezado `Content-Type: application/json`. Para que *los errores* de validación y de autenticación también salgan en JSON (y no como una redirección o una página HTML), el cliente debe enviar el encabezado `Accept: application/json`; el dashboard lo hace en cada llamada.

4.3.4. Códigos de estado usados

Código	Significado	Dónde ocurre
200	OK	Consultas y actualizaciones exitosas.
201	Created	Registro, creación de usuarios y de roles.
400	Bad Request	Un administrador intenta eliminarse a sí mismo.
401	Unauthorized	Credenciales incorrectas en el login; token ausente, inválido, vencido o revocado.
403	Forbidden	Usuario autenticado sin el rol requerido (<code>RoleMiddleware</code>).
404	Not Found	<code>findOrFail</code> sobre un usuario inexistente.
422	Unprocessable Entity	Falla de validación de datos de entrada.
429	Too Many Requests	Más de 5 intentos de login por minuto.
500	Internal Server Error	Error no controlado (mensaje genérico en producción).

4.4. Autenticación JWT con Laravel Passport

4.4.1. Por qué Passport

Laravel Passport implementa un servidor OAuth2 completo y emite tokens de acceso en formato JWT firmados con criptografía asimétrica RSA. Frente a un token opaco simple, esto tiene dos ventajas: el token es verificable sin consultar secretos compartidos y su firma no puede falsificarse sin la clave privada, que solo conoce el servidor.

4.4.2. Configuración del guard

En `config/auth.php` se declara el guard `api` usando el driver de Passport:

```
'guards' => [  
    'web' => [  
        'driver' => 'session',  
        'provider' => 'users',  
    ],  
    'api' => [  
        'driver' => 'passport',  
        'provider' => 'users',  
    ],  
],
```

Listing 24: `config/auth.php` (fragmento)

Un *guard* define *cómo* se autentica cada petición. El guard `web` usa sesiones y cookies (para vistas tradicionales); el guard `api` lee el token `Bearer` del encabezado. El middleware `auth:api` de las rutas selecciona explícitamente este segundo guard.

4.4.3. Claves RSA

Passport firma los tokens con un par de claves generado mediante:

```
php artisan passport:keys
```

Listing 25: Generación de claves y cliente de Passport

Se crean dos archivos en `storage/`:

- `oauth-private.key`: clave **privada**. Firma los tokens. Es secreta.
- `oauth-public.key`: clave **pública**. Permite verificar las firmas.

Los permisos verificados en el servidor son `-rw----` (modo 600) para ambas, con propietario `www-data`: solo ese usuario puede leerlas o escribirlas. Además, el archivo `.gitignore` contiene la regla `/storage/*.key`, por lo que **nunca se suben al repositorio**.

4.4.4. Tablas de Passport

Tabla	Función
<code>oauth_access_tokens</code>	Un registro por cada token emitido: usuario, cliente, <code>revoked</code> (booleano), <code>expires_at</code> . Es la tabla que consulta Passport para saber si un token fue revocado.
<code>oauth_refresh_tokens</code>	Tokens de renovación (flujos OAuth2 completos).
<code>oauth_auth_codes</code>	Códigos de autorización (flujo <i>authorization code</i>).
<code>oauth_clients</code>	Aplicaciones cliente registradas.
<code>oauth_device_codes</code>	Flujo de autorización por dispositivo.

4.4.5. Ciclo de vida de un token

1. **Emisión.** En el login, `createToken()` inserta una fila en `oauth_access_tokens` y genera el JWT firmado con la clave privada.
2. **Uso.** El cliente envía el JWT en `Authorization: Bearer`. El guard de Passport verifica la firma con la clave pública, comprueba `exp` y consulta que la fila correspondiente no esté revocada.
3. **Vigencia.** 12 horas desde el login (ver siguiente apartado).
4. **Revocación.** En el logout, `revoke()` marca `revoked = 1`. El token deja de funcionar de inmediato.

4.4.6. Cómo se valida cada petición

El middleware `auth:api` realiza las siguientes comprobaciones antes de ejecutar el controlador. Si cualquiera falla, la respuesta es 401:

1. El encabezado `Authorization` existe y comienza con `Bearer`.
2. El token tiene una estructura JWT válida (tres segmentos).
3. La **firma RS256** coincide con la clave pública (el token no fue alterado).
4. La fecha de expiración (`exp`) no ha pasado.
5. El token no está marcado como revocado en la base de datos.
6. El usuario al que pertenece (`sub`) todavía existe.

```
curl -i http://127.0.0.1:8000/api/v1/user -H "Accept: application/json"
# HTTP/1.1 401 Unauthorized {"message":"Unauthenticated."}

curl -i http://127.0.0.1:8000/api/v1/user \
-H "Accept: application/json" -H "Authorization: Bearer token.falso
.123"
```

```
# HTTP/1.1 401 Unauthorized {"message":"Unauthenticated."}
```

Listing 26: Comportamiento sin token y con token manipulado

5. Requerimientos de Seguridad Computacional

Este capítulo describe cada medida de seguridad solicitada, **dónde** se encuentra en el proyecto, **cómo** funciona y **cómo verificarla**. Cuando una medida depende de la configuración del servidor de producción (y no del código del repositorio) se indica de forma explícita.

5.1. 1. Cifrado de contraseñas

Se utilizó el algoritmo **Bcrypt** con *salt* autogenerado utilizando el `Facade Hash::make($password)` de Laravel. Ninguna contraseña es reversible ni se almacena en texto plano.

5.1.1. Dónde se aplica

- `AuthController::register: 'password' =>Hash::make($request->password).`
- `AdminController::createUser:` igual, al crear un usuario desde el panel.
- `RolesAndPermissionsSeeder:` las cuentas de demostración se crean con `Hash::make`.
- `User::casts():` el *cast* `'password' =>'hashed'` actúa como red de seguridad adicional.
- `AuthController::login:` la verificación se realiza con `Hash::check`, nunca comparando textos.

5.1.2. Cómo funciona Bcrypt

1. Se genera un **salt** aleatorio de 16 bytes para cada contraseña. Dos usuarios con la misma contraseña obtienen hashes completamente distintos, lo que anula las *rainbow tables*.
2. Se aplica el algoritmo con un **factor de costo** (por defecto 12 en Laravel, es decir 2^{12} iteraciones), que lo hace deliberadamente lento y encarece los ataques de fuerza bruta contra una base filtrada.
3. El resultado ocupa 60 caracteres y contiene versión, costo, salt y hash en un solo texto.

```
$2y$12$Kq3b8vYw1a1k7mQe0XJmU0e9d2zq6o3b0wV3k0rC0fQ0k9c2m1u5S
|   |   |                               |
|   |   |                               '-- hash resultante (31 caracteres)
|   |   '----- salt aleatorio (22 caracteres)
```

```
| ,----- costo (2^12 iteraciones)
| ,----- version del algoritmo
```

Listing 27: Anatomía de un hash Bcrypt almacenado en la columna password

5.1.3. Verificación

```
sqlite3 database/database.sqlite "select email, substr(password,1,7)
from users;"
# admin@admin.com|$2y$12$
# editor@editor.com|$2y$12$
```

Listing 28: Comprobar que la base no contiene contraseñas en claro

El prefijo \$2y\$ confirma que son hashes Bcrypt. Además, la contraseña mínima de 8 caracteres exigida en el registro reduce el riesgo de contraseñas triviales.

5.2. 2. Comunicación Cifrada (HTTPS/TLS)

Se estableció en las directivas de despliegue que todo el tráfico se forzará por HTTPS. La API interceptará el protocolo para evitar secuestros de sesión (Man-in-the-Middle).

5.2.1. Por qué es indispensable

El token JWT viaja en el encabezado `Authorization`. Si la comunicación fuera HTTP, cualquier equipo entre el cliente y el servidor (por ejemplo, una red WiFi pública) podría leer el token y usarlo para suplantar al usuario mientras no expire. TLS cifra todo el canal: contraseñas del login, tokens y respuestas JSON.

5.2.2. Cómo se implementa en producción

HTTPS se termina normalmente en el servidor web (Nginx o Apache) con un certificado, por ejemplo de Let's Encrypt. Configuración típica con Nginx:

```
server {
    listen 80;
    server_name ejemplo.com;
    return 301 https://$host$request_uri;    # fuerza HTTPS
}

server {
    listen 443 ssl http2;
    server_name ejemplo.com;

    ssl_certificate      /etc/letsencrypt/live/ejemplo.com/fullchain.pem
    ;
    ssl_certificate_key  /etc/letsencrypt/live/ejemplo.com/privkey.pem;
    ssl_protocols       TLSv1.2 TLSv1.3;
```

```
add_header Strict-Transport-Security "max-age=31536000;
    includeSubDomains" always;

root /var/www/llm-chat/public;
index index.php;
location / { try_files $uri $uri/ /index.php?$query_string; }
}
```

Listing 29: Ejemplo de configuración Nginx: redirección a HTTPS y HSTS

Del lado de la aplicación, Laravel puede forzar que todas las URLs generadas usen el esquema seguro. Se hace en `AppServiceProvider::boot()`:

```
use Illuminate\Support\Facades\URL;

public function boot(): void
{
    if (app()->environment('production')) {
        URL::forceScheme('https');
    }
}
```

Listing 30: Forzar esquema https en producción (a agregar en `AppServiceProvider::boot()`)

Estado en el repositorio

El código actual no incluye aún esa línea ni un archivo de configuración del servidor; el servidor de desarrollo (`php artisan serve`) trabaja sobre HTTP en `127.0.0.1:8000`, lo cual es aceptable únicamente en un entorno local. La medida se cumple al desplegar con la configuración de ejemplo anterior. Ver recomendaciones en el capítulo ??.

5.3. 3. Gestión segura de tokens JWT

- **Expiración:** Originalmente se configuró un tiempo corto. Posteriormente se ajustó para propósitos de evaluación.
- **Firma:** El token es firmado internamente usando claves RSA (RS256) generadas con el comando `passport:keys`. Dichas claves (`oauth-private.key`) tienen permisos estrictos 600 y no se almacenan en el control de versiones.

5.3.1. Expiración

La vigencia se fija en el login:

```
$tokenResult = $user->createToken('Personal Access Token');
$token = $tokenResult->token;
// Extendemos el tiempo de expiración para que no caduque rápido
$token->expires_at = now()->addMinutes(720); // 12 horas
$token->save();
```

Listing 31: Establecer la expiración a 12 horas (`AuthController::login`)

Un token de vida corta limita el daño si es robado. El valor de 12 horas se eligió para comodidad durante la evaluación del proyecto (el dashboard no necesita reautenticarse constantemente); en un entorno productivo se recomienda reducirlo (15 a 60 minutos) y usar tokens de refresco.

5.3.2. Firma y permisos de las claves

```
$ ls -l storage/oauth-*.key
-rw----- 1 www-data www-data 3322 Oct  5 02:09 storage/oauth-private.
key
-rw----- 1 www-data www-data  812 Oct  5 02:09 storage/oauth-public.
key

$ grep key .gitignore
/storage/*.key
```

Listing 32: Estado verificado de las claves en el servidor

Interpretación de `-rw----`: el propietario (`www-data`, el usuario del servidor web) tiene lectura y escritura; el grupo y los demás usuarios del sistema no tienen ningún acceso. La clave privada de 3322 bytes corresponde a una clave RSA de 4096 bits.

5.3.3. Revocación

Además de la expiración, los tokens pueden cancelarse antes de tiempo. El logout lo hace con `$request->user()->token()->revoke()`. Con esto se evita que un token “olvidado” siga siendo válido horas después de que el usuario cerró sesión.

5.3.4. Almacenamiento del token en el cliente

El dashboard guarda el token en `localStorage` (`jwt_token`) y lo elimina al cerrar sesión o al recibir un 401. Es un compromiso habitual en SPAs. Su riesgo principal es que un ataque XSS podría leerlo, por lo que la protección contra XSS (siguiente secciones) es especialmente importante. Una alternativa más robusta sería una cookie `HttpOnly; Secure; SameSite=Strict`.

5.4. 4. Autorización verificada en el servidor

Independientemente de que el frontend oculte visualmente botones mediante CSS/JS, la seguridad principal recae en el servidor mediante los Middlewares `role` y `permission`.

5.4.1. El principio

Nunca se confía en el cliente. Ocultar un botón en la interfaz es una comodidad de usabilidad, no un control de seguridad: cualquier persona puede abrir las herramientas del navegador, o usar `curl`, y enviar la petición directamente. Por eso cada ruta sensible valida el rol en el servidor, con información de la base de datos.

5.4.2. Implementación

- **role:** alias de `RoleMiddleware` registrado en `bootstrap/app.php`. Es el que usan hoy todas las rutas `/admin/*`.
- **permission:** el paquete `Spatie` incluye su propio middleware de permisos. Como los permisos (*Lectura*, *Escritura*, *Eliminación*) ya están cargados en las tablas, puede aplicarse a rutas concretas cuando las funciones de negocio (conversaciones, edición de contenido) se implementen. Para habilitarlo basta registrar el alias en `bootstrap/app.php`:

```
$middleware->alias([
    'role'      => \App\Http\Middleware\RoleMiddleware::class,
    'permission' => \Spatie\Permission\Middleware\PermissionMiddleware::class,
]);

// Uso en rutas:
Route::delete('/contenido/{id}', ...) -> middleware('permission:
    Eliminacion');
```

Listing 33: Registro del alias permission de Spatie (extensión prevista)

5.4.3. Prueba de que el ocultamiento visual no basta

Aunque el frontend no muestre el panel a un usuario regular, si este conoce la URL e intenta la llamada, el servidor lo detiene:

```
curl -i -X DELETE http://127.0.0.1:8000/api/v1/admin/users/1 \
-H "Authorization: Bearer $TOKEN_DE_UN_NO_ADMIN" -H "Accept:
    application/json"
# HTTP/1.1 403 Forbidden
# {"message":"Forbidden - Insufficient permissions"}
```

5.5. 5. Validación y Sanitización de Entradas

Laravel protege contra Inyección SQL mediante sentencias preparadas de PDO en su ORM. Para ataques XSS y validaciones masivas, se usó `$request->validate()` para sanear tipos de datos en todos los Controladores.

5.5.1. Reglas aplicadas por endpoint

Endpoint	Reglas de validación
register	name: required string max:255; email: required email unique:users; password: required min:8.

Endpoint	Reglas de validación
login	email: required email; password: required.
admin/users (POST)	Nombre, correo único, contraseña min:8, role: exists:roles,name.
admin/users/{id} (PUT)	Campos opcionales (sometimes); correo único salvo el propio; rol existente.
admin/users/{id}/role	role: required string exists:roles,name.
admin/roles	name: unique:roles,name; permissions.*: exists:permissions,name.

5.5.2. Defensas combinadas

1. **Inyección SQL:** sentencias preparadas (ver capítulo anterior). Las reglas `exists` y `unique` también se ejecutan con parámetros enlazados.
2. **Asignación masiva (*mass assignment*):** \$fillable en User y AuditLog. Aunque un atacante agregue campos adicionales al JSON (por ejemplo `is_admin: true`), Eloquent los ignorará. En `updateUser` se usa explícitamente `$request->only(['name', 'email', 'password', 'role'])`.
3. **Escalada de privilegios por registro:** el registro público siempre asigna Usuario Regular; el rol no se toma de la petición.
4. **XSS:** el backend devuelve datos como JSON (`application/json`), no HTML, por lo que el navegador no los ejecuta como código. En el cliente, la tabla del dashboard se construye con `createElement` y asignación de texto (ver capítulo 6); esta práctica evita interpretar HTML malicioso contenido, por ejemplo, en el nombre de un usuario.

5.5.3. Ejemplo de rechazo por validación

```
curl -i -X POST http://127.0.0.1:8000/api/v1/register \
-H "Accept: application/json" -H "Content-Type: application/json" \
-d '{"name":"X","email":"no-es-correo","password":"123"}'
```

```
{
  "message": "The email field must be a valid email address. (and 1
    more error)",
  "errors": {
    "email": ["The email field must be a valid email address."],
    "password": ["The password field must be at least 8 characters."]
  }
}
```

Listing 34: Respuesta 422

5.6. 6. Protección contra ataques de fuerza bruta

Se protegió el endpoint `/login` con el middleware `throttle:5,1`. Si un atacante falla la contraseña más de 5 veces en un minuto, recibe un código **429 Too Many Requests**.

5.6.1. Dónde está

```
Route::post('/login', [AuthController::class, 'login'])
    ->middleware('throttle:5,1'); // Limit to 5 requests per minute
    against brute force
```

Listing 35: routes/api.php

5.6.2. Cómo funciona

`throttle:5,1` significa *5 intentos cada 1 minuto*. Laravel identifica al solicitante (por IP, o por usuario si está autenticado), cuenta sus peticiones en la caché (en este proyecto el almacén `cache` configurado en la tabla de base de datos) y, al exceder el límite, responde 429 con el encabezado `Retry-After` que indica cuántos segundos esperar. Un ataque de diccionario que podría probar miles de contraseñas por minuto queda reducido a cinco.

5.6.3. Combinación con otras defensas

El límite se complementa con (a) el mensaje de error único del login, que evita enumerar usuarios; (b) el costo de Bcrypt, que hace caro cada intento y (c) el registro de auditoría de cada inicio de sesión exitoso, que permite detectar accesos extraños.

```
for i in 1 2 3 4 5 6; do
    curl -s -o /dev/null -w "intento $i -> %{http_code}\n" \
        -X POST http://127.0.0.1:8000/api/v1/login \
        -H "Accept: application/json" -H "Content-Type: application/json" \
        -d '{"email":"admin@admin.com","password":"incorrecta"}'
done
# intento 1 -> 401 ... intento 5 -> 401
# intento 6 -> 429
```

Listing 36: Demostración: el sexto intento es bloqueado

5.7. 7. Registro de auditoría con trazabilidad

El modelo `AuditLog` guarda el `user_id`, `ip_address`, `action` y `timestamp`. Estos registros solo pueden insertarse mediante lógica en el backend y no pueden ser alterados por nadie, manteniendo su peso legal y confidencial.

5.7.1. Qué eventos quedan registrados

Evento	Acción registrada	Origen
Registro de usuario	<i>Registro de usuario</i>	<code>AuthController::register</code>
Inicio de sesión exitoso	<i>Inicio de sesión</i>	<code>AuthController::login</code>
Cierre de sesión	<i>Cierre de sesión</i>	<code>AuthController::logout</code>
Crear / editar / eliminar usuario	<i>Crear Usuario, Actualizar Usuario, Eliminar Usuario</i>	<code>AdminController</code>
Asignar rol / crear rol	<i>Asignar Rol, Crear Rol</i>	<code>AdminController</code>
Cualquier petición autenticada	Método y ruta (p. ej. <i>GET api/v1/admin/users</i>)	<code>AuditMiddleware</code>

5.7.2. Cómo consultar la bitácora

```
curl -s http://127.0.0.1:8000/api/v1/admin/audit-logs \
-H "Authorization: Bearer $TOKEN_ADMIN" -H "Accept: application/json"
```

```
[
  {"id": 12, "user_id": 1, "action": "GET api/v1/admin/users",
   "ip_address": "127.0.0.1", "details": "[]",
   "created_at": "2026-10-05T23:10:41.000000Z"},
  {"id": 11, "user_id": 1, "action": "Inicio de sesión",
   "ip_address": "127.0.0.1", "details": "Inicio de sesión exitoso",
   "created_at": "2026-10-05T23:10:38.000000Z"}
]
```

Listing 37: Ejemplo de registros devueltos

Los registros se devuelven ordenados del más reciente al más antiguo gracias a `AuditLog::latest()`.

5.7.3. Sobre la inmutabilidad y la confidencialidad

- **Inmutabilidad a nivel de aplicación:** no existe ninguna ruta, controlador ni método del código que actualice o elimine filas de `audit_logs`. Solo se inserta y se lee.
- **Confidencialidad:** la lectura está restringida al rol Administrador; un Editor o un Usuario Regular reciben 403.
- **Datos sensibles:** el middleware elimina `password` y `password_confirmation` antes de guardar el cuerpo de la petición.
- **Conservación histórica:** la llave foránea con `nullOnDelete` mantiene los registros de un usuario aunque su cuenta sea borrada.

5.8. 8. Manejo seguro de CORS

El archivo `config/cors.php` restringe qué orígenes pueden consumir la API, previniendo que scripts maliciosos alojados en otros dominios roben la sesión o datos.

5.8.1. Qué es CORS

Los navegadores aplican la *política del mismo origen*: una página en `https://sitio-malo.com` no puede leer la respuesta de una petición a `https://api.ejemplo.com` a menos que el servidor lo autorice con encabezados `Access-Control-Allow-Origin`. CORS (*Cross-Origin Resource Sharing*) es el mecanismo para otorgar esas autorizaciones de manera controlada.

5.8.2. Configuración

Laravel incluye el middleware `HandleCors`, que lee la configuración de `config/cors.php`. Una configuración restrictiva sería:

```
return [
    'paths' => ['api/*'],
    'allowed_methods' => ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
    'allowed_origins' => ['https://ejemplo.com'], // solo el frontend
    'allowed_origins_patterns' => [],
    'allowed_headers' => ['Authorization', 'Content-Type', 'Accept'],
    'exposed_headers' => [],
    'max_age' => 0,
    'supports_credentials' => false,
];
```

Listing 38: `config/cors.php` (configuración restrictiva recomendada)

Se publica con `php artisan config:publish cors`.

Estado en el repositorio

En el árbol actual **no existe** el archivo `config/cors.php`; Laravel usa entonces su configuración predeterminada, que permite cualquier origen en las rutas `api/*`. Como el dashboard se sirve desde el *mismo* dominio que la API, funciona sin necesidad de CORS, pero la restricción descrita arriba aún debe publicarse y fijarse a los dominios reales antes del despliegue. La política más segura combina esto con que el token viaja en un encabezado (no en una cookie), lo que ya impide el abuso por CSRF.

5.9. 9. Principio de Mínimo Privilegio

Un editor no puede ver la tabla de usuarios. Un usuario no puede borrar contenido. Esto se logró definiendo reglas atómicas en Spatie y limitando los accesos a lo que estrictamente requiere cada función.

5.9.1. Cómo se aplica en el proyecto

1. **Rol por defecto mínimo.** Todo registro nuevo es `Usuario Regular` (solo *Lectura*).
2. **Permisos atómicos.** Lectura, Escritura y Eliminación son permisos separados; cada rol recibe únicamente los necesarios.
3. **Rutas de administración exclusivas.** Las siete rutas `/admin/*` requieren el rol `Administrador`. El Editor no puede ver usuarios ni la bitácora.
4. **Un solo rol por usuario administrado.** `syncRoles` evita acumular privilegios de forma accidental.
5. **Protección contra autoeliminación.** El administrador no puede borrar su propia cuenta.
6. **Respuesta mínima.** Los campos `password` y `remember_token` están en `$hidden`, así que nunca se exponen en el JSON de usuarios.

5.9.2. Comparación de capacidades por rol

Acción	Usuario Regular	Editor	Administrador
Iniciar sesión y ver su perfil	Sí	Sí	Sí
Listar usuarios	No (403)	No (403)	Sí
Crear/editar/eliminar usuarios	No (403)	No (403)	Sí
Asignar roles y crear roles	No (403)	No (403)	Sí
Consultar bitácora de auditoría	No (403)	No (403)	Sí

5.10. 10. Manejo de errores sin fuga de información

Se configuró el entorno `APP_ENV=production` y `APP_DEBUG=false`. En caso de que falle la base de datos o el código, el usuario no verá trazas de pila (Stack Traces) ni configuraciones internas; solo recibirá un error **HTTP 500** genérico.

5.10.1. Por qué importa

Con `APP_DEBUG=true`, Laravel muestra páginas de error detalladas que incluyen rutas de archivos del servidor, fragmentos de código, consultas SQL y valores del entorno. Para un atacante, esa información es un mapa del sistema. En producción solo debe verse un mensaje genérico mientras el detalle se guarda en `storage/logs/laravel.log`, accesible solo al personal técnico.

5.10.2. Configuración requerida en el archivo .env de producción

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://ejemplo.com
LOG_LEVEL=warning
```

Listing 39: .env de producción (no versionado)

Tras cambiarlo conviene ejecutar `php artisan config:cache` para que Laravel use la configuración ya compilada.

Estado actual del entorno de desarrollo

El archivo `.env` de este equipo de desarrollo tiene `APP_ENV=local` y `APP_DEBUG=true`, lo cual es correcto para programar y depurar. La directiva `production/false` aplica al despliegue final y debe verificarse antes de publicar. El archivo `.env` está excluido del control de versiones (`.gitignore`), por lo que ningún secreto se sube al repositorio.

5.10.3. Comportamiento esperado

```
HTTP/1.1 500 Internal Server Error
Content-Type: application/json

{"message": "Server Error"}
```

Listing 40: Respuesta de error en producción

Los errores 401, 403, 404, 422 y 429 descritos antes son errores *esperados* y usan mensajes breves y controlados que no revelan detalles internos.

6. Frontend Web y Experiencia de Usuario

El cliente web es una aplicación de una sola página (SPA) autocontenida en el archivo `public/admin.html`. Su objetivo es demostrar el consumo de la API de forma segura y proveer una interfaz visual de gestión sin recurrir a frameworks pesados (React, Vue) ni a motores de plantillas acoplados al backend (Blade).

6.1. Diseño y Estética (Glassmorphism)

El requerimiento exigía un diseño moderno y fluido. Se optó por la tendencia *Glassmorphism*, que simula paneles de cristal esmerilado flotando sobre un fondo dinámico o colorido.

6.1.1. Implementación visual

El efecto cristal se logra mediante la propiedad CSS `backdrop-filter`, que desenfoca el contenido que está *detrás* del elemento, y un fondo con transparencia (`rgba`):

```
.glass-panel {  
  background: rgba(255, 255, 255, 0.1);  
  backdrop-filter: blur(12px);  
  -webkit-backdrop-filter: blur(12px);  
  border: 1px solid rgba(255, 255, 255, 0.2);  
  box-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.3);  
  border-radius: 16px;  
  padding: 2rem;  
}
```

Listing 41: Estilo base para los paneles translúcidos

El fondo de la página (`body`) tiene un gradiente animado que se mueve lentamente para darle vida a la interfaz. Las tipografías provienen de Google Fonts (*Inter* para legibilidad en tablas y *Outfit* para encabezados).

6.2. Estructura Lógica del Cliente

El archivo contiene tres secciones principales:

1. **Estructura (HTML):** Define dos grandes contenedores (vistas): el formulario de login (`#login-view`) y el panel de administración (`#dashboard-view`). También incluye un modal (`#crud-modal`) oculto por defecto para la creación y edición de usuarios.
2. **Estilos (CSS):** Contiene variables para la paleta de colores y las reglas para la tabla, botones y animaciones (`@keyframes`).
3. **Comportamiento (JS):** Un bloque de *Vanilla JavaScript* (sin dependencias como jQuery o Axios) que maneja el ciclo de vida de los datos y las interacciones.

6.3. Consumo Seguro de la API REST

Toda comunicación usa la función asíncrona `fetch` nativa del navegador.

6.3.1. Gestión del Token en el cliente

Al cargar la página, el script intenta recuperar un token previo:

```
const API_URL = '/api/v1';  
let jwtToken = localStorage.getItem('jwt_token') || null;
```

Listing 42: Inicialización del token JWT

En el login exitoso, la API responde con el JSON y el script guarda el token:


```
jwtToken = data.access_token;
localStorage.setItem('jwt_token', jwtToken);
```

Listing 43: Almacenamiento local del token

Para *cualquier* otra petición al servidor, el token se adjunta en los encabezados. Si el servidor responde 401 (token vencido o revocado), el cliente lo borra y vuelve al login:

```
const response = await fetch(`${API_URL}/admin/users`, {
  method: 'GET',
  headers: {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'Authorization': 'Bearer ${jwtToken}'
  }
});
```

Listing 44: Envío del token en la lectura de usuarios

6.3.2. Prevención de XSS en la construcción del DOM

Uno de los vectores de ataque más comunes en SPAs es el XSS (Cross-Site Scripting) basado en el DOM. Si un administrador inserta un usuario con el nombre `<script>alert(1)</script>` y el frontend simplemente concatena ese valor en una cadena HTML con `innerHTML`, el código malicioso se ejecutará.

El dashboard está protegido contra esto de dos formas:

1. La API rechaza la entrada si la validación estricta la detecta.
2. En la vista, la creación de elementos se hace mediante métodos seguros del DOM (`document.createElement` y `textContent`).

6.4. Manejo de Errores y Respuesta visual

El cliente provee retroalimentación inmediata usando alertas emergentes en la parte superior:

```
function showAlert(msg, isError = false) {
  alertBox.textContent = msg;
  alertBox.className = isError ? 'alert-error' : 'alert-success';
  alertBox.style.display = 'block';
  setTimeout(() => { alertBox.style.display = 'none'; }, 4000);
}
```

Listing 45: Función para mostrar mensajes de estado

Cuando falla una petición debido a permisos insuficientes (403), el catch captura el estado y alerta al usuario:

```
if (response.status === 403) {
  showAlert('No tienes permiso de Administrador.', true);
}
```

```
    return;  
}
```

6.5. Pruebas de la API vía CLI

Dado que la API es desacoplada, no depende de este frontend. Puede ser consumida desde una terminal o herramientas como Postman.

```
export TOKEN=$(curl -s -X POST http://127.0.0.1:8000/api/v1/login \  
-H "Accept: application/json" -H "Content-Type: application/json" \  
-d '{"email":"admin@admin.com","password":"password123"}' | grep -o '  
    "access_token":"[^"]*' | cut -d'"' -f4)
```

Listing 46: 1. Autenticarse e inferir token

```
curl -s http://127.0.0.1:8000/api/v1/user \  
-H "Authorization: Bearer $TOKEN" -H "Accept: application/json"
```

Listing 47: 2. Consultar el propio perfil protegido

7. Arquitectura SaaS Multi-Tenant y App Móvil Kotlin

7.1. 1. Objetivo General de la Plataforma

Desarrollar y operar una plataforma SaaS (Software as a Service) multi-empresa que permita a diversos negocios y empresas gestionar, personalizar y publicar digitalmente su catálogo de productos y precios. Cada empresa cuenta con un código QR único que los clientes finales pueden escanear desde sus dispositivos móviles o mediante la aplicación nativa Android escrita en Kotlin.

7.2. 2. Jerarquía de Roles y Matriz de Control de Acceso

La plataforma implementa un control de acceso basado en roles (RBAC) con aislamiento lógico estricto:

Rol	Nivel de Jerarquía			Permisos y Alcance
admin	Administrador General SaaS			Acceso total a todas las empresas, gestión de usuarios globales, planes de suscripción y bitácora general de auditoría.
empresa	Panel	Empresarial	Multi-Tenant	Gestión exclusiva de los productos, categorías, precios e imagen corporativa de su propia empresa (<code>company_id</code>). Aislamiento de datos estricto.
cliente	Usuario Final / Cliente Mobile			Consulta del catálogo público vía escaneo de código QR o consumo de APIs REST desde la aplicación móvil nativa Android en Kotlin.

Principio de Aislamiento Multi-Tenant

Ninguna empresa puede consultar, modificar o eliminar los productos o categorías pertenecientes a otra empresa. El backend valida en cada petición que el parámetro `company_id` corresponda al tenant del usuario autenticado.

7.3. 3. Modelo de Datos y Diagrama de Tablas MySQL

El esquema relacional en MySQL está estructurado para soportar multi-tenancy eficiente y auditoría continua:

```
-- Tabla de Empresas (Tenants)
CREATE TABLE companies (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  slug VARCHAR(255) UNIQUE NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  phone VARCHAR(50) NULL,
  plan ENUM('Basico', 'Plus', 'Premium') DEFAULT 'Basico',
  status ENUM('Activo', 'Inactivo') DEFAULT 'Activo',
  created_at TIMESTAMP, updated_at TIMESTAMP
);

-- Tabla de Usuarios con vinculo a Empresa
CREATE TABLE users (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  role ENUM('admin', 'empresa', 'cliente') DEFAULT 'cliente',
  company_id BIGINT NULL,
  FOREIGN KEY (company_id) REFERENCES companies(id) ON DELETE SET
  NULL
);
```

```

-- Tabla de Categorías por Empresa
CREATE TABLE categories (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  company_id BIGINT NOT NULL,
  name VARCHAR(255) NOT NULL,
  description TEXT NULL,
  FOREIGN KEY (company_id) REFERENCES companies(id) ON DELETE CASCADE
);

-- Tabla de Productos y Precios
CREATE TABLE products (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  company_id BIGINT NOT NULL,
  category_id BIGINT NULL,
  name VARCHAR(255) NOT NULL,
  description TEXT NULL,
  price DECIMAL(10,2) NOT NULL,
  is_active BOOLEAN DEFAULT TRUE,
  FOREIGN KEY (company_id) REFERENCES companies(id) ON DELETE CASCADE
,
  FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE SET
  NULL
);

```

Listing 48: Estructura principal de tablas en MySQL

7.4. 4. Comparativa de Tecnologías Utilizadas

Tecnología	Componente	Propósito y Justificación Técnica
Laravel 11	Backend Framework	Proporciona la arquitectura del servidor, enrutamiento web, controladores API REST y la capa de autenticación con Laravel Passport/Sanctum.
MySQL 8.0	Motor de Base de Datos	Almacena relacionalmente las empresas, productos, categorías, usuarios y registros inmutables de auditoría.
Docker & n8n	Orquestación e IA	Servidor en contenedores que ejecuta el motor de automatización n8n para aplicar filtros inteligentes con IA sobre los productos.
Kotlin (Android)	Aplicación Móvil Nativa	Consume los endpoints JSON del catálogo y permite a los clientes escanear códigos QR con la cámara del celular.
Laravel Blade	Frontend Panel Web	Renderiza el panel de control administrativo responsivo con Tailwind CSS y contadores de sesión en tiempo real.

7.5. 5. Flujo de Integración API con App Móvil Kotlin

La comunicación entre la aplicación móvil Android y el servidor Laravel se realiza mediante peticiones HTTPS/HTTP con respuestas en formato JSON estructurado:

```
GET /api/v1/catalogo/tech-corp HTTP/1.1
Host: 127.0.0.1:8000
Accept: application/json
```

Listing 49: Peticion HTTP enviada por la App Android Kotlin al escanear el QR

```
{
  "status": "success",
  "empresa": {
    "nombre": "Tech Corp Inc.",
    "slug": "tech-corp",
    "qr_url": "http://127.0.0.1:8000/q/tech-corp"
  },
  "categorias": [
    {
      "name": "Software & Licencias",
      "products": [
        { "id": 1, "name": "Licencia Anual LLM Chat Pro", "price":
          "4999.00" }
      ]
    }
  ]
}
```

Listing 50: Respuesta JSON recibida por la App Kotlin